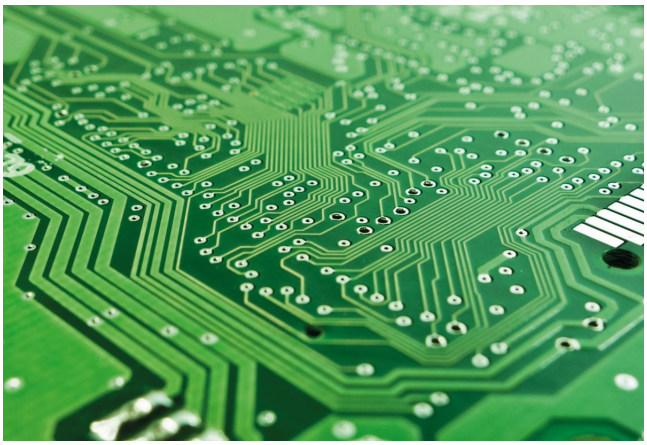
UNIVERSITÉ DE LIÈGE

LIÈGE université
Sciences Appliquées

ELEN0448-A-a
Applied Electricity and Electronics
Signal generator



Authors :
CASARIL Maud *s202160*
DALEM Pierre *s191571*
DUVIVIER Arnaud *s203457*
GODFIRNON Florentin *s201328*
VANDERHEYDEN Julien *s203734*
WEBER Tom *s203806*

Professors :
JM. Redouté
P. Vanderbemden

Année Universitaire : 2023-2024

1 Schematics diagrams of the circuit

The schematic diagram of the circuit is given in Fig. 1. To fully understand all this circuit, it is important to make a brief description of every component to explain the choices made.

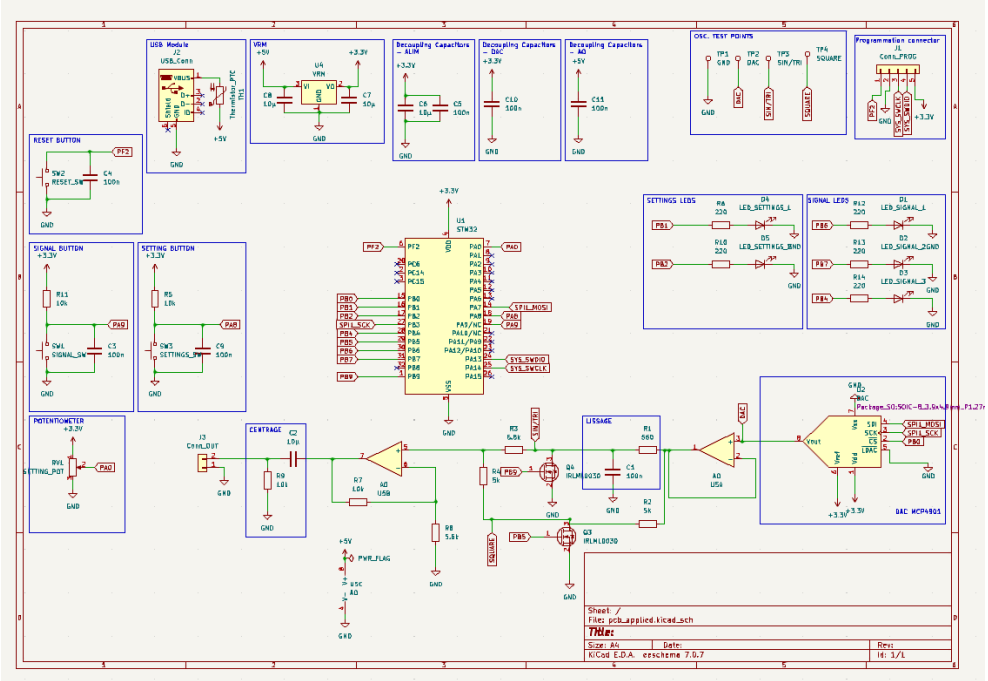


FIGURE 1 – Schematic diagram of the circuit

1.1 Power Supply

The power supply is made with a USB connector, bringing a 5V signal on our PCB. This connector is followed by a thermistor, to avoid big currents. Then, a voltage regulator converts the 5V signal into a 3.3V signal (input voltage of the micro-controller and the DAC). Two 10 µF capacitors are set at the input and output of the regulator, as written in its data-sheet. The 3.3V signal is finally stabilized by several decoupling capacitors, placed closed to the corresponding components.

1.2 Microcontroller

The main component is the micro-controller. This central component with its internal connections enables the various others components to interact with each other.

1.3 Digital to Analog Converter

A digital to Analog Converter (also called DAC) is a component which enables to convert a digital signal into an analog signal. Here, the input digital signal comes from the micro-controller and represents the amplitude of the signal. There are three connections with the micro-controller :

- `Serial Clock Input` is connected to the clock of the SPI with PB3.
- `Serial Data Input` which receives the digital information and is connected to the SPI with PA7.
- `Chip Select` which allows information to flow when in low mode and is connected to PB0.

The power supplied to the DAC is 3.3V. To prevent any damage to the DAC and avoid parasitic signals, we chose to connect the output of the DAC with an operational amplifier with unity gain. This connection enables us to have an impedance adaptation with the rest of the circuit. The AO insulator is supplied with 5V.

1.4 Sinus/triangle and square way

After the analog signal has passed through the AO insulator, there are two different paths : one of them is for a square signal and the other one is for a sinus or triangle signal. To send the signal

to the correct path, two transistors are used. The transistor lets the signal flow to grid if the signal type is the same as that of the branch and in the opposite case, the transistor is in drain-source mode so the signal goes to the ground.

Signals which control the mode of the transistors come from PB9 and PB5. If the transistor of the sinus/triangle path lets the signal go, the signal can flow in a RC filter which is used to smooth out the signal. This low pass filter only allows low frequency signals from 0Hz to its cut-off frequency to pass while blocking those that are any higher. The cut-off frequency is given by : $f_c = \frac{1}{RC} = \frac{1}{560 \times 100 \times 10^{-9}} = 1787,14Hz$.

The two branches then go in an addition operational amplifier, summing up the two signals (one of the two is always grounded). It permits the squared signal to avoid the smoothing filter. In the two branches, there are resistances which offer a potential difference between the two branches. According to this, the value of these resistors have to be large enough to create a potential difference.

1.5 Centering and amplification

The outgoing signal from the sinus/triangle and the squared branch is attenuated after going in the resistance. To have the largest range of amplitude possible, we use an operational amplifier with a gain equal to two. To do so, the feedback resistance is twice as large as the resistance connected to the ground. Thus, the output signal is twice as large as the input signal. AO supply is 5V. After this, the signal goes in a filter composed of a resistance and a capacitor. The capacitor enables to remove the DC component of the signal which will center the signal. It is important to understand that the output of the DAC is strictly positive so the signal have to be centered.

1.6 Buttons

To enable the user to interact with the signal generator, some buttons are useful. We choose to use three buttons :

- `Reset button` : it enables the user to reset the signal which means that the generator restarts to generate signal, it is connected to PF2.
- `Signal button` : it enables to change the type of the signal generated and is connected to PA9.
- `Setting button` : it enables to change the type of parameter controlled by the potentiometer. The parameter of the signal can be the frequency or the amplitude. This button is connected to PA8.

From an electrical point of view, it is important to note that there always is a capacitor in parallel to the button. It can be explained by the fact the mechanical contacts do not close instantly, but bounce several times before stabilizing when the button is pressed. These capacitors serve as a hardware filter, and avoids the bounces.

1.7 LEDs

Several LEDs are placed on the PCB to indicate :

1. the "mode" of the potentiometer : amplitude or frequency
2. the output : sin/tri/sqr

As seen on Fig. 1, to avoid excessive current, there is a resistance before every LED.

1.8 Programmation connector

Some pins of the micro-controller are used to program it, so these pins are placed at a specific place of the PCB, and soldered to a connector. As we do not make any use of the UART function, we chose not to add the two pins related to that function in our connector.

1.9 Test points

Some test points were placed so the signal can be seen from different places of the treatment. Thanks to these, if there is an issue, it can be debugged. We choose to put a test point on the ground, a test point at the exit of the DAC (TP2), a test point after a the MOSFET for the sinus or triangle signal and a last test point for the square signal.

2 Layout of the PCB

The layout of the PCB can be seen on the Fig. 2.

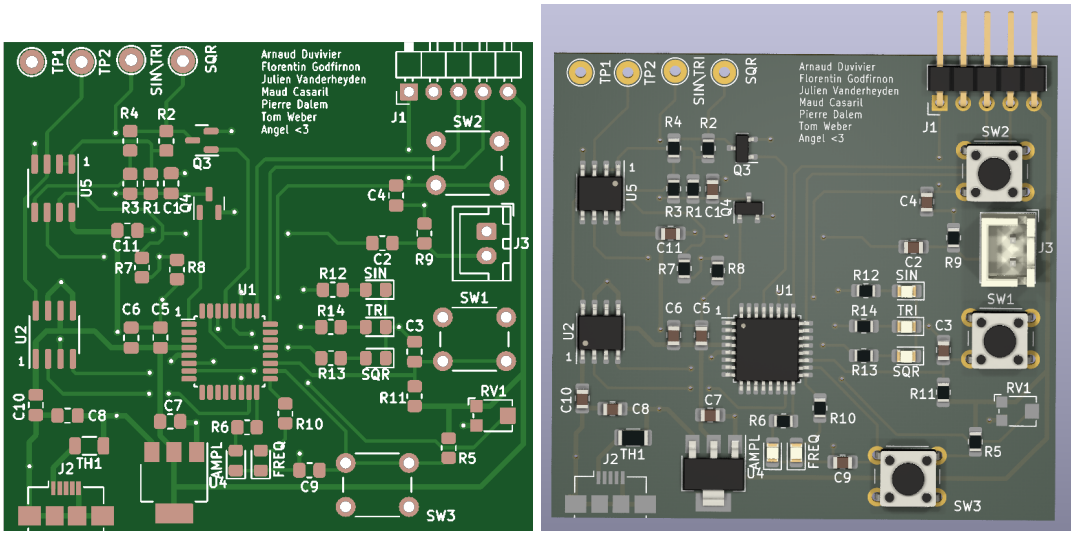


FIGURE 2 – PCB views

It can be seen that the USB connector and the programmer are put on the boundaries of the PCB to facilitate the connections. The different interactions with the user (switch, potentiometer and leds) are placed together for ease of use.

The decoupling capacitors were placed near the components to which they are linked.
The placement of the various components was designed to limit the length of the wires and to maximize the relationship between the components. The goal was also to minimize the number of tracks on the back of the PCB to avoid a maximum to split the ground plane.

The main difficulty was to place all the components to have enough space for the tracks. Especially around the microprocessor, as it is connected to a lot of parts of the circuit, it should be well placed and it took several tries.

3 Debugging the code and validation of the circuit

Before building the PCB, all our features were tested on a breadboard. The code was checked and tested this way. Thanks to this, the debug part was pretty easy because we were able to change the hardware as the software fast and to see the result on the oscilloscope. We built the code in parallel with the hardware part, and we did this progressively, beginning with a very simple circuit and adding features step by step, when we was sure that the previous part was OK. In parallel with the breadboard, the electric schematic was done on KiCad.

The validation of the circuit was made thanks to a lot of tests on different circuits and with different values for the components. Some tests failed, others were pretty good. At the end, we created a complete circuit that is the result of the combination of the small circuits that we tested step by step.

For example, the first problem we had to deal with was to send the SPI frame correctly. As we were trying to send 01010101 01010101, it took a while to notice on the oscilloscope that our program was sending the two 8-bit binary words in the wrong order : this problem led to the function `convertForSPI` that performs the binary operations on these words.

Furthermore, the fact that the square signal is by-passing the smoothing filter in order to keep a beautiful square shape was not straightforward : we have tried several different configurations before reaching the final one (voltage additioner with transitors grounding one of the two branches) and then we also had to try a lot of different values for the resistors in order to have the maximal

output voltage for the two signales, while not reaching the saturation of the amplifier.

At the end of the implementation of the code, we noticed that the use of the potentiometer lead our project to a small issue. For certain values of the reading of the potentiometer, the signal that we generated had a kind of freeze. We were able to see it on the oscillospe with the "stop" mode. With many experiments, we saw that this problem happened when the value of the potentiometer is not a multiple of the size of the table were we store the signal. So, to solve this, we force the potentiometer to take a value in a table with only the different multiples of the size of the table of the signal (360 in our project), with biggest value being one tenth of this size. This solved the problem pretty well.

A last and more annoying issue was the wrong footprint of the DAC and AO packages. It was still possible to tinker with that PCB but we decided to order a new one. Given the time spent on the project, we preferred to have a nice PCB that we could be proud of.

4 Structure of the code

In the main loop of our code, the array `tab` is filled with the corresponding values (depending on the waveform, the frequency and the amplitude). Then, an interrupt is triggered on overflow of the `TIMER1`, sending an SPI frame to the DAC. We continuously iterate over this array, unless the user triggers another interrupt (by pressing one button to change the waveform or by turning the potentiometer to change the frequency/amplitude). In that case, the array is re-computed.

4.1 Array generation

The array `tab` is filled by the function `generate_signal`, which has two main functionalities : turning on and off the corresponding pins (transistors and leds associated to the waveform), and calling the function related to the wave form. These are the three functions `generate_triangle`, `generate_triangle` and `generate_square`, responsible for filling the array with the good values, depending on the different parameters of the signal (frequency and amplitude). Here are their prototypes :

```
void generate_sinus(uint8_t tab[],  uint16_t len_tab, uint16_t amplitude, uint16_t freq)
void generate_triangle(uint8_t tab[],  uint16_t len_tab, uint16_t amplitude, uint16_t freq)
void generate_square(uint8_t tab[],  uint16_t len_tab, uint16_t amplitude, uint16_t freq, double duty_cycle )
void generate_signal(uint16_t amplitude , uint16_t freq, uint8_t mode)
```

4.2 TIMER1 overflow interrupts

In order to reach higher frequencies, we placed the interrupt handler in the `.it` file. The related function is called `TIM16_IRQHandler` and its only action is to raise the flag used in the main loop to send the SPI frame.

4.3 SPI frames generation

We made a function called `ConvertForSPI`, which is responsible for converting the value we want to send to the DAC in a binary frame following the SPI documentation.
After that, this frame is sent by these three steps : turning down the CS pin, sending the SPI frame using the `HAL_SPI_Transmit` function, and turning on the CS pin.

4.4 Button interrupts

The interruptions triggered by a change on a GPIO pin can be configured in the built-in function `HAL_GPIO_EXTI_Falling_Callback`. Depending on the source pin, this function changes the value of one of the following global variables : either `mode` (storing the waveform) or `change_amplitude` (storing if the potentiometer is acting on the frequency or the amplitude). In that case, the function also modifies the level of the pins to display in which mode we are (frequency or amplitude).

4.5 ADC handling

Finally, the ADC is the last functionality we implemented. The value of the potentiometer is recovered with the three functions `HAL_ADC_Start`, `HAL_ADC_PollForConversion` and `HAL_ADC_GetValue`.

The program then checks if the value read has changed by a significant value (in comparison to a certain threshold). If so, the value between 0 and 4096 is mapped into the corresponding min/max values (depending on the value of the global variable `change_amplitude`), and a new `tab` array is generated with the new parameters by the functions seen at the first point of this section.

5 Future improvements

One possible improvement we could bring to the project is to use transistors instead of resistors. In our case we only have 5 LEDs so the heat generated by the resistors is negligible. However if we were to have many more LEDs, heating could cause a problem.

To protect the micro-controller from receiving a higher voltage than it can withstand, we could place a diode in between the transistor and the micro-controller. This would offer an additional safety measure to protect the chip if the transistor was to ever short circuit. This can also be avoided if a FET is used as the base remains isolated.

Another major improvement we could bring to our project is to increase the speed. This can be achieved by sending 16 bits in one go instead of two times 8 bits. In our function `ConvertForSPI` we shift certain bits to obtain correct input/output matching. If we avoid this issue and send 16 correct bits in one go, we can save a significant amount of time by skipping the passage trhough our conversion function. All the information would also be sent in a single communication. Saving time for theses operations would allow us to increase the frequency.

On the PCB, some little mistakes remain. First of all, two test points are named tp1 and tp2. It should be written GND and DAC. A title should be added to each switch to distinguish them one from the other. Some components should be a little further apart to make soldering easier. This is practical knowledge we gained through our first soldering experience as is something will keep in mind for the future. From an aesthetic point of view, some changes to the alignment of certain components could make the design more fluid. We could maybe also place the USB connector and programmer port on the same side.

More globally, we have to check more in detail the PCB file before ordering. Indeed, some mistakes were made on the footprints of the DAC and the AO. Because of that, we had to order a new PCB otherwise, we would have had to made something not clean. We also chose the wrong footprint for the MOSFETs.

The last issue that we encountered was a soldering mistake. A resistance was not well soldered and then the signal was stopped there.
Now that all these mistakes have been done, we know at what we have to pay attention to.

6 Conclusion

In conclusion we would like to mention that this project was particularly fun. We were able to quickly apply and transfer concepts learnt in the classroom. As well as getting some hands on work with the soldering of all the components on our PCB. Even with all the trials and mishaps we have encountered, we are proud of what we have achieved. We believe that all these skills will help us in our future electrical engineering careers!